

Morphological Operators

```
In [8]: import cv2
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

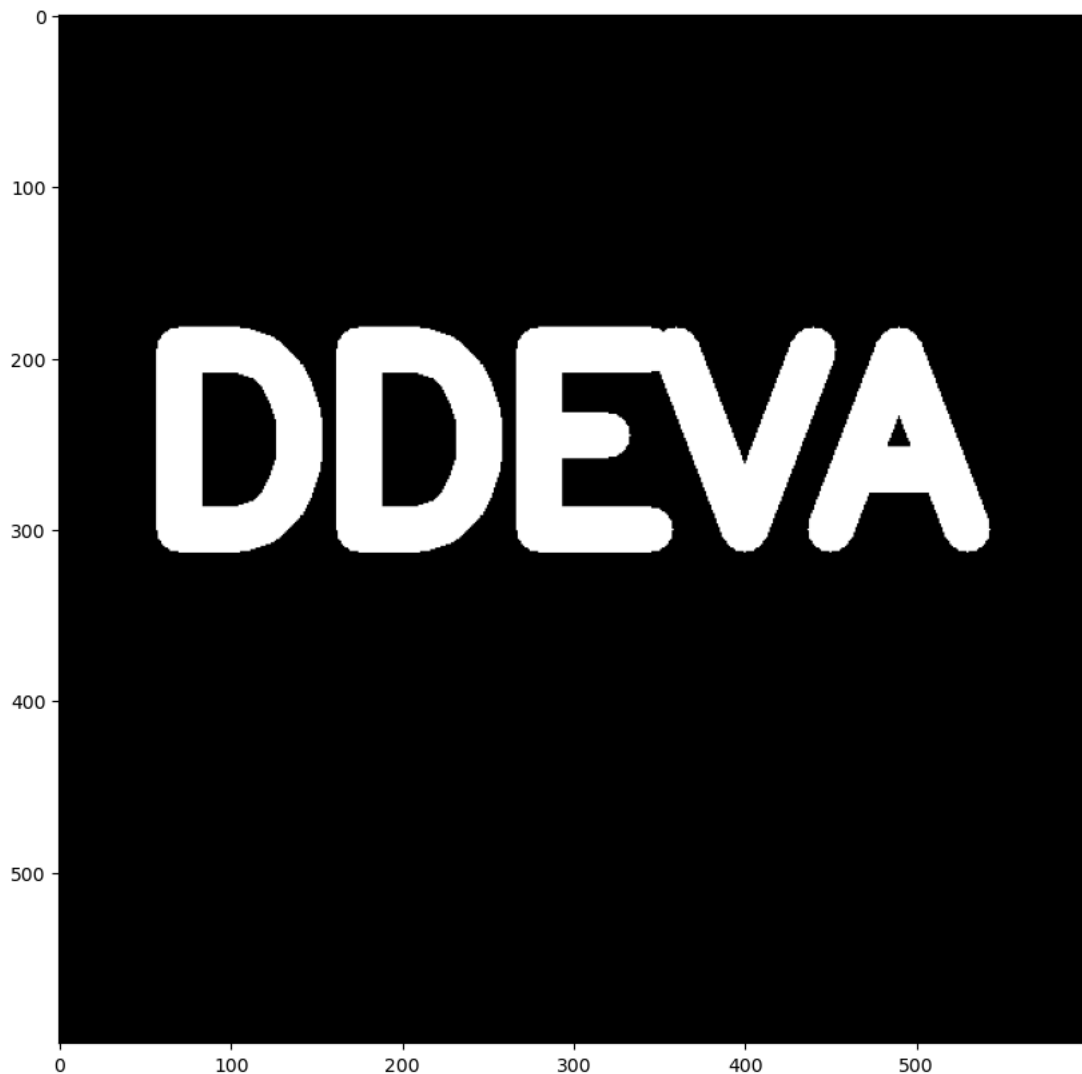
```
In [9]: # Replace 'ABCDE' with your name (only the first 5 characters)
```

```
In [14]: def load_img():
    blank_img = np.zeros((600,600))
    font = cv2.FONT_HERSHEY_SIMPLEX
    cv2.putText(blank_img, text='DDEVA', org=(50,300), fontFace=font, fontScale= 5, color=(255,255,255), thickness=25, lineType=cv2.LINE_AA)
    return blank_img
```

```
In [15]: def display_img(img):
    fig = plt.figure(figsize=(12,10))
    ax = fig.add_subplot(111)
    ax.imshow(img, cmap='gray')
    plt.show()
```

```
In [16]: img = load_img()
```

```
In [17]: display_img(img)
```



Erosion

Erodes away boundaries of foreground objects. Works best when foreground is light color (preferably white) and background is dark.

```
In [ ]: # Create a 5x5 kernel filled with ones and display it using dtype=np.uint8
```

```
In [19]: kernel = np.ones((5,5),dtype=np.uint8)
```

```
In [ ]:
```

```
In [ ]:
```

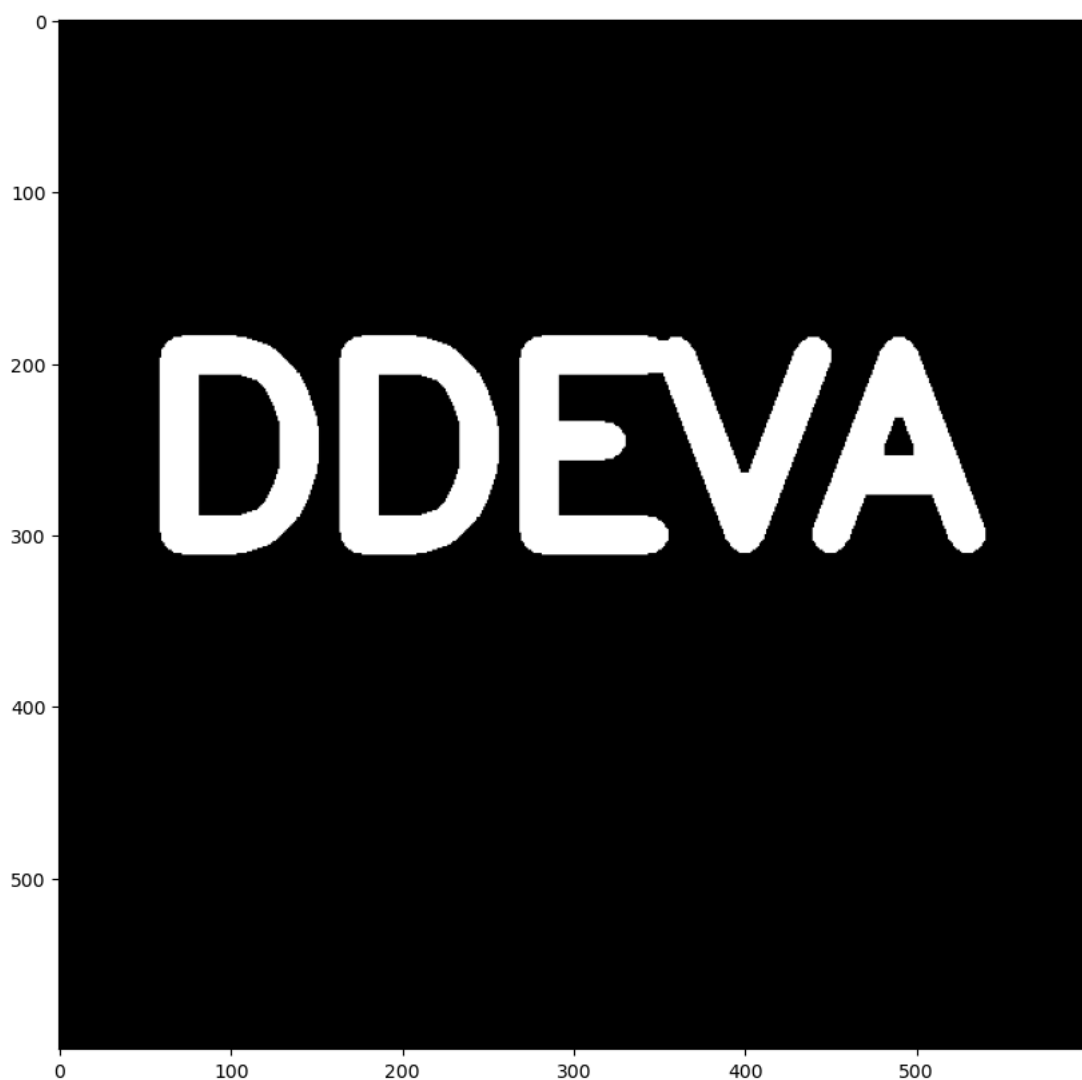
```
In [20]: kernel
```

```
Out[20]: array([[1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1],
               [1, 1, 1, 1, 1]], dtype=uint8)
```

```
In [31]: # Apply erosion on the image using the specified kernel with one iteration and store the result in erosion1 using the cv2.ero
```

```
In [22]: erosion1 = cv2.erode(img, kernel, iterations=1)
```

```
In [23]: display_img(erosion1)
```



Opening

Opening is erosion followed by dilation. Useful in removing background noise!

```
In [24]: img = load_img()
```

```
In [ ]: # creat white noise with randint(Low=0,high=3,size=(600,600))
```

```
In [25]: white_noise = np.random.randint(low=0, high=3, size=(600, 600), dtype=np.uint8)
```

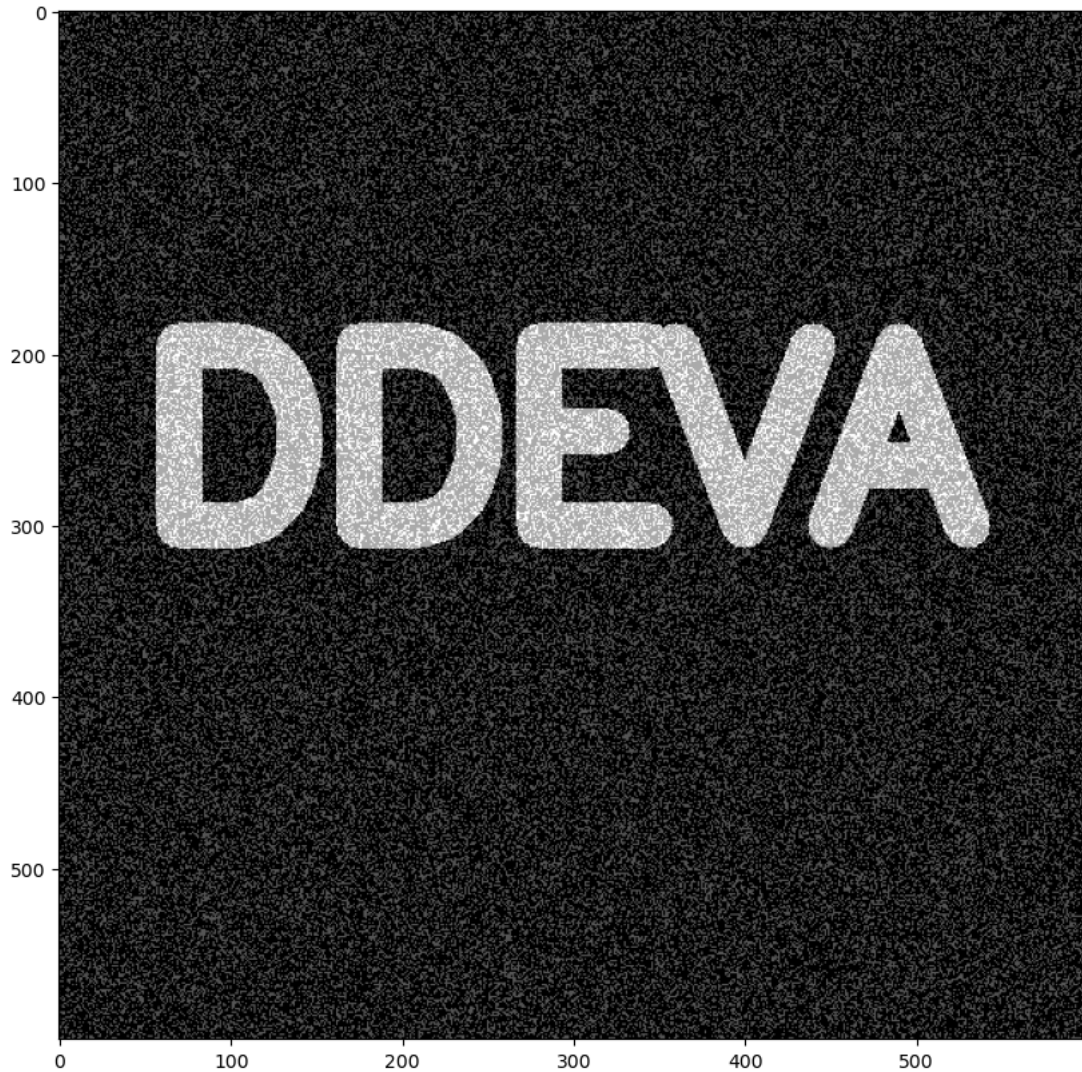
```
In [27]: white_noise = white_noise * 127
```

```
In [28]: white_noise = white_noise*255
```

```
In [29]: white_noise = np.clip(white_noise, 0, 255).astype(np.uint8)
```

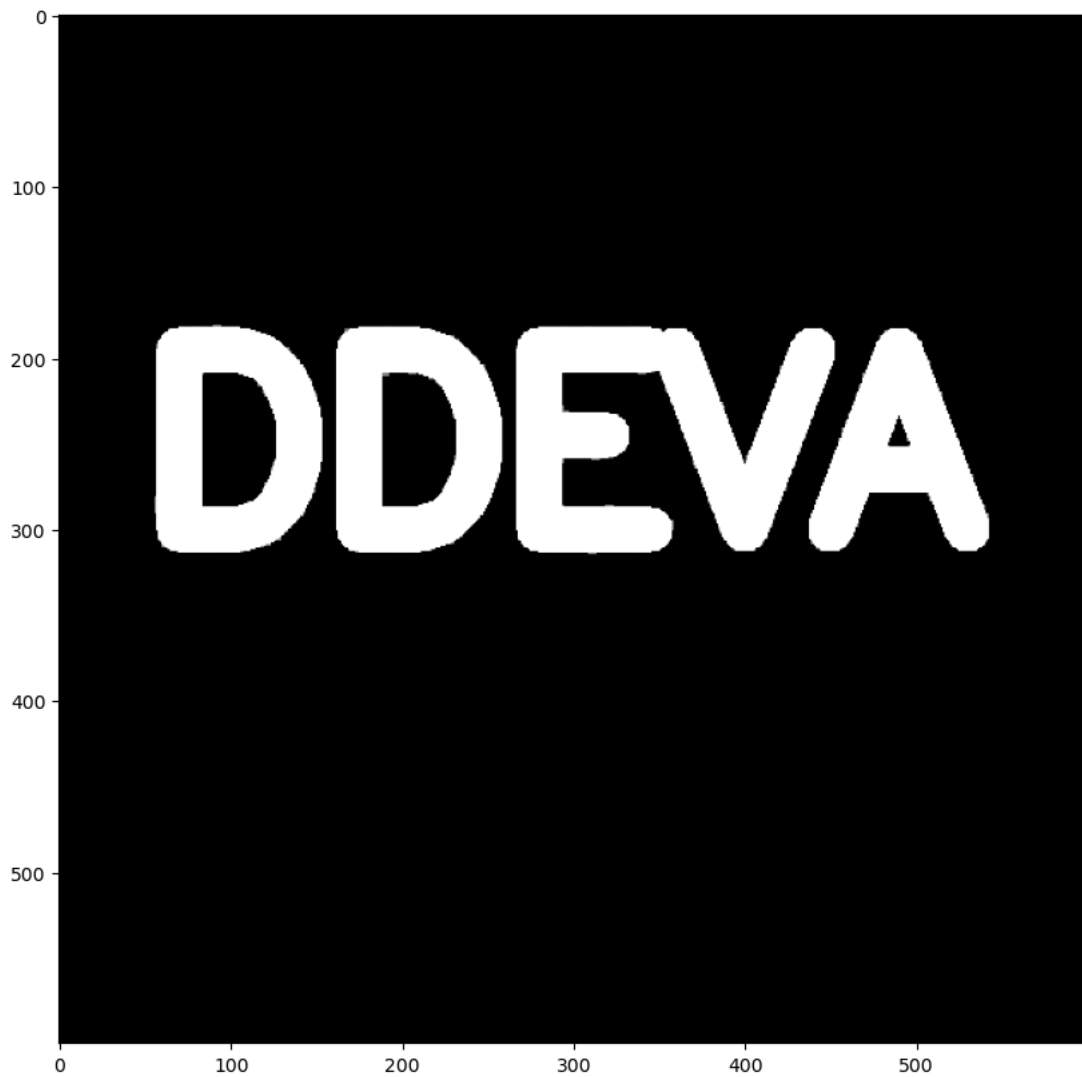
```
In [30]: noise_img = white_noise+img
```

```
In [31]: display_img(noise_img)
```



```
In [32]: opening = cv2.morphologyEx(noise_img, cv2.MORPH_OPEN, kernel)
```

```
In [33]: display_img(opening)
```



Closing

Useful in removing noise from foreground objects, such as black dots on top of the white text.

```
In [183]: # creat white noise with randint(low=0,high=2,size=(600,600))
```

```
In [34]: img = load_img()
```

```
In [35]: black_noise = np.random.randint(low=0,high=2,size=(600,600))
```

```
In [36]: black_noise
```

```
Out[36]: array([[1, 0, 1, ..., 0, 1, 0],
                [0, 0, 1, ..., 1, 1, 1],
                [1, 0, 0, ..., 0, 0, 0],
                ...,
                [1, 1, 0, ..., 1, 0, 0],
                [1, 1, 1, ..., 1, 0, 0],
                [1, 0, 1, ..., 0, 0, 1]])
```

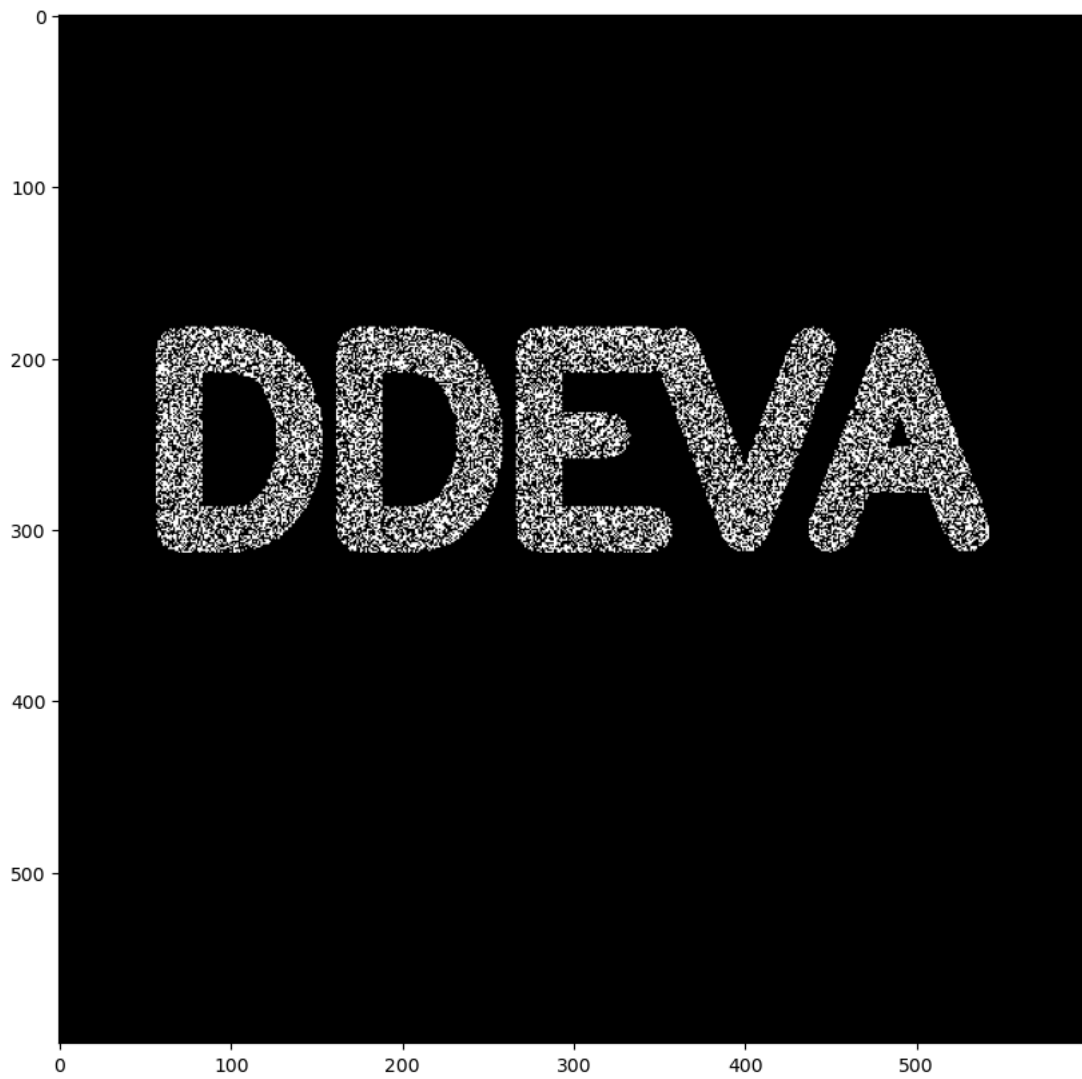
```
In [37]: black_noise= black_noise * -255
```

```
In [38]: black_noise_img = img + black_noise
```

```
In [39]: black_noise_img = np.clip(black_noise_img, 0, 255).astype(np.uint8)
```

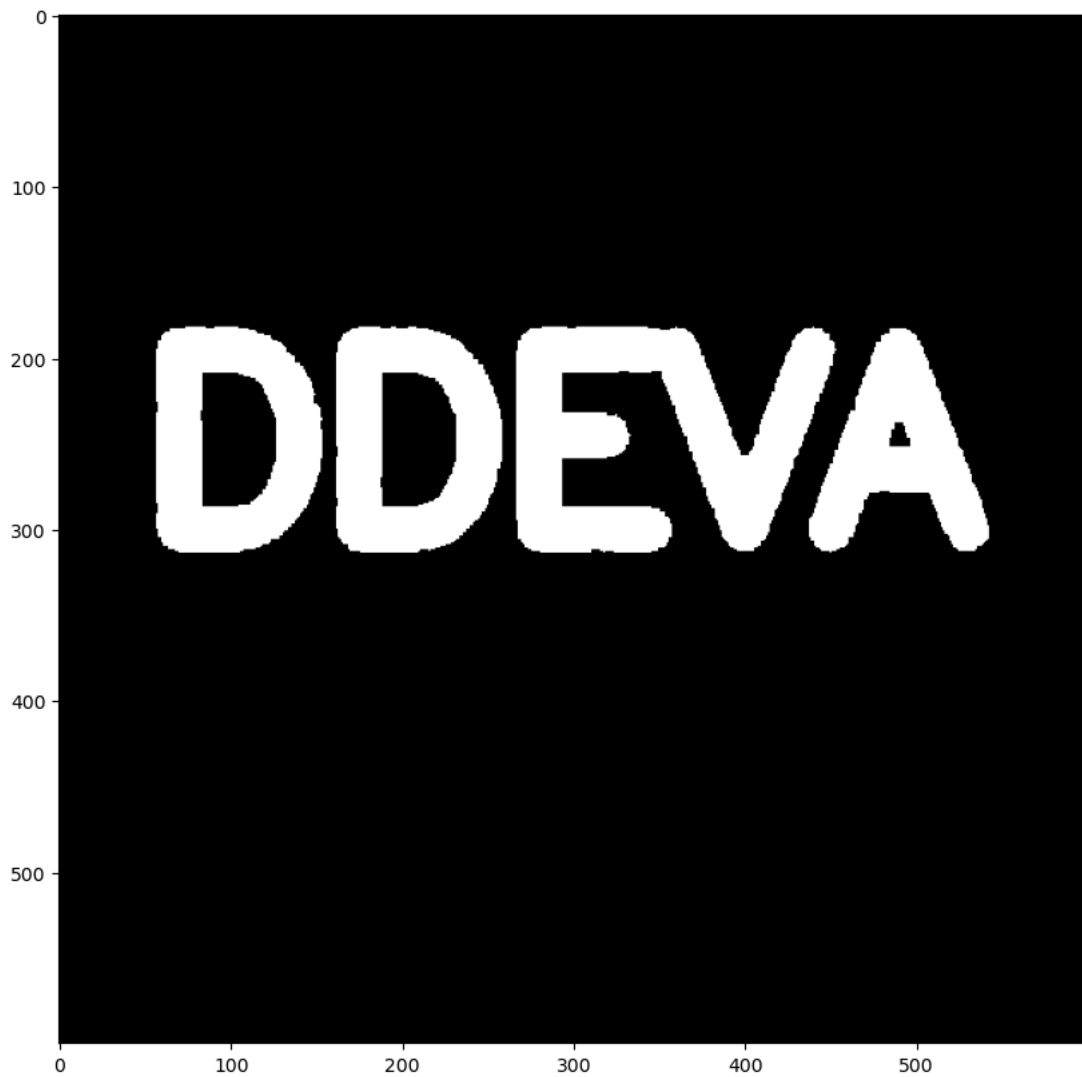
```
In [43]: black_noise_img[black_noise_img== -255] = 0
```

```
In [44]: display_img(black_noise_img)
```



```
In [45]: closing = cv2.morphologyEx(black_noise_img, cv2.MORPH_CLOSE, kernel)
```

```
In [46]: display_img(closing)
```

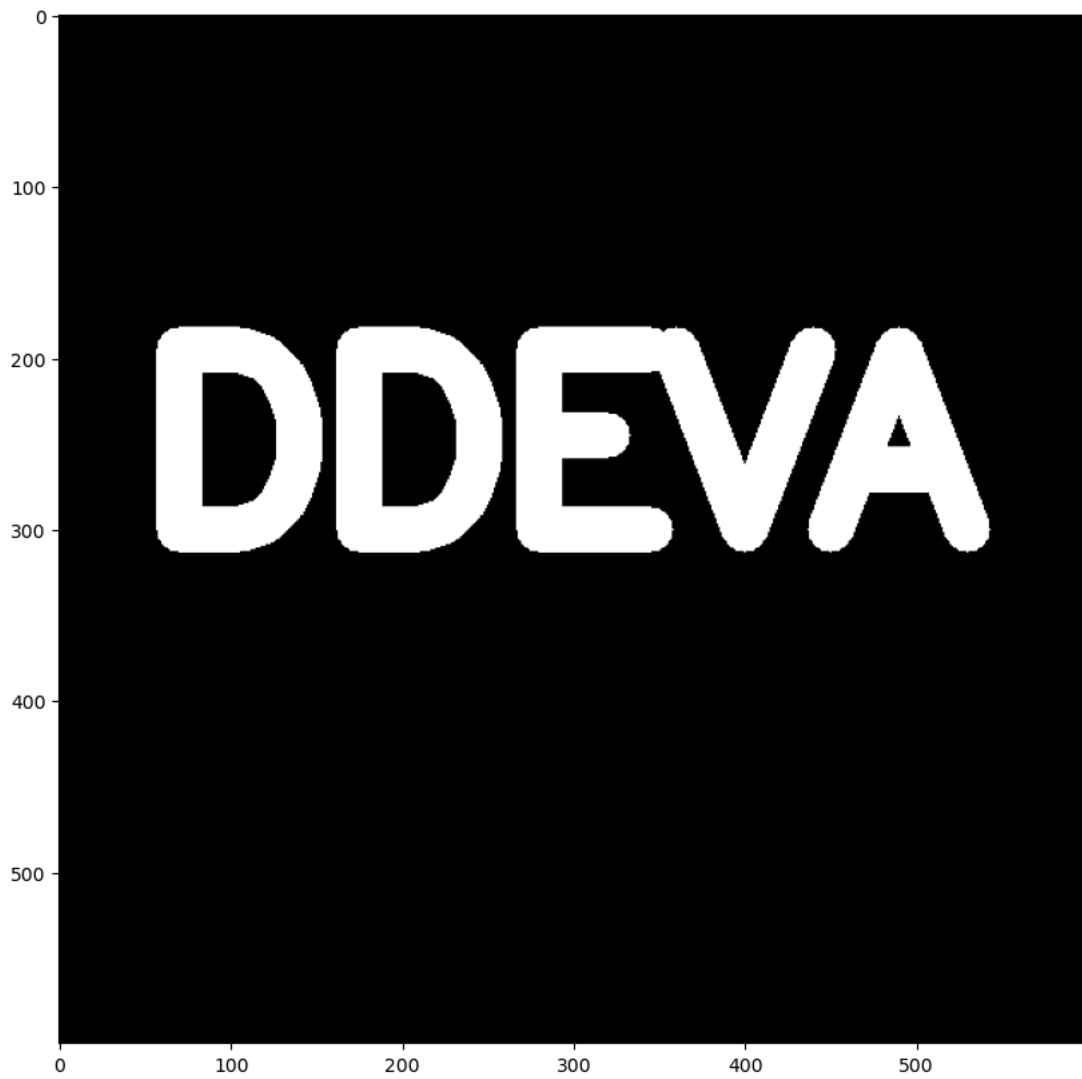


Morphological Gradient

Difference between dilation and erosion of an image.

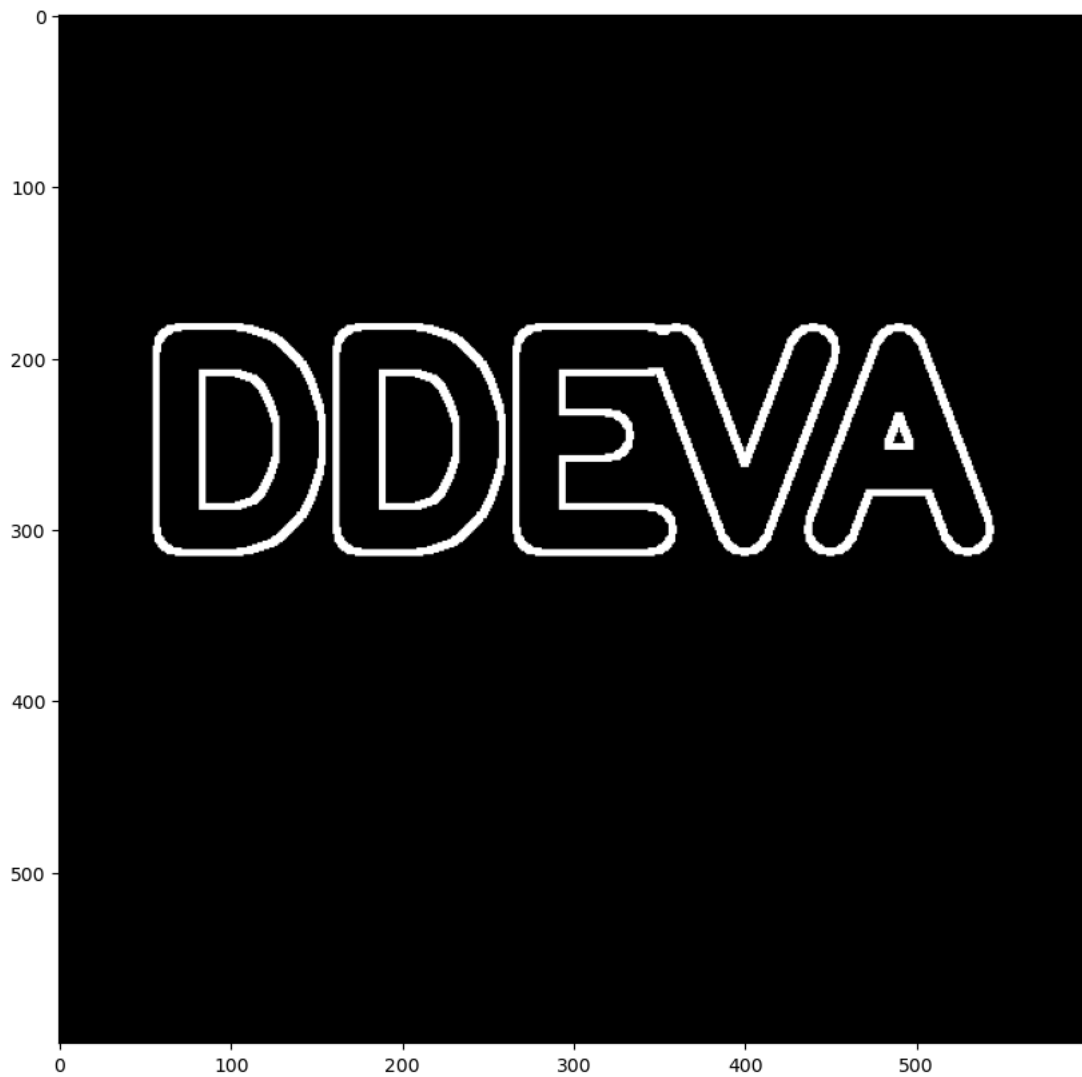
```
In [47]: img = load_img()
```

```
In [48]: display_img(img)
```



```
In [49]: gradient = cv2.morphologyEx(img, cv2.MORPH_GRADIENT, kernel)
```

```
In [50]: display_img(gradient)
```



Great work!